Notes:

- Don't forget HW0 if you haven't already completed it!
- No class next Tuesday
- HW1 due October 2
- HW2 posted Thursday

Last class:

- more Python basics
  - data types and operations
  - functions
  - standard library
  - importing modules

Today:

- version control and git

To create a repository from scratch, make a new directory and run `git init` in it.

```
scottmac@kaczmarz:~/CMSC6950$ mkdir CMSC6950_Fall2025_example
scottmac@kaczmarz:~/CMSC6950$ cd CMSC6950_Fall2025_example/
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/scottmac/CMSC6950/CMSC6950_Fall2025_example/.git/
```

Notice that message refers to a `.git` subdirectory`:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ ls .git/
branches  config  description  HEAD  hooks  info  objects  refs
```

THis is where git keeps all of its local information

Start a new branch:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git checkout -b main
Switched to a new branch 'main'
```

Check status of repo with

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Now create a file:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ nano myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        myname.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Now git knows that we've done something, but in a file that we haven't (yet) told git to track. Let's tell it to do something:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   myname.txt
```

Now lists "changes to be committed". Git records our changes only when we commit them to a repository.

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git commit -m"Add myname.txt"
[main (root-commit) 69db617] Add myname.txt
 1 file changed, 2 insertions(+)
 create mode 100644 myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
nothing to commit, working tree clean
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git log
commit 69db617546d4211d75acaa93cb1180999486a5dd (HEAD -> main)
Author: Scott MacLachlan <smaclachlan@mun.ca>
Date:   Tue Sep 23 10:41:22 2025 -0230

    Add myname.txt
```

Each commit should have a message, a short description of what has been done. You will see many *terrible* commit messages (e.g., "foo" or "doing some things" or "k"). **DON'T DO THIS!!**

Now edit file:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ nano myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   myname.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

Again see "no changes added to commit"

By default, git does not keep track of anything you do in a repo. General pattern is:

1. Edit files
2. `git add` on each edited file
3. `git commit` on your changes

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git commit -m"Adding university"
[main 48fdbd0] Adding university
 1 file changed, 1 insertion(+)
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git log
commit 48fdbd0944395a142df853c4460a8412a21d4267 (HEAD -> main)
Author: Scott MacLachlan <smaclachlan@mun.ca>
Date:   Tue Sep 23 10:47:21 2025 -0230

    Adding university

commit 69db617546d4211d75acaa93cb1180999486a5dd
Author: Scott MacLachlan <smaclachlan@mun.ca>
Date:   Tue Sep 23 10:41:22 2025 -0230

    Add myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
nothing to commit, working tree clean
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$
```

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ nano myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git commit -m"Adding title"
[main 1413f06] Adding title
 1 file changed, 1 insertion(+), 1 deletion(-)
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git log
commit 1413f064c3bfa01ead2afed259181dc47e36aa72 (HEAD -> main)
Author: Scott MacLachlan <smaclachlan@mun.ca>
Date:   Tue Sep 23 10:48:16 2025 -0230

    Adding title

commit 48fdbd0944395a142df853c4460a8412a21d4267
Author: Scott MacLachlan <smaclachlan@mun.ca>
Date:   Tue Sep 23 10:47:21 2025 -0230

    Adding university

commit 69db617546d4211d75acaa93cb1180999486a5dd
Author: Scott MacLachlan <smaclachlan@mun.ca>
Date:   Tue Sep 23 10:41:22 2025 -0230

    Add myname.txt
```

Can use `git commit -a -m"message"` to skip adding files one-by-one and add all files that have been modified, but this is dangerous. May add too many files (backup files, compiled output, ...)

Try to not commit compiled artifacts (log files, object files, compiled pdf files) to our repos, because these are difficult to sync across machines (different architectures)

Use `git diff` to see changes before you commit them:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ nano myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git diff
diff --git a/myname.txt b/myname.txt
index 24415be..38bbd7f 100644
--- a/myname.txt
+++ b/myname.txt
@@ -1,3 +1,3 @@
 Professor Scott MacLachlan
-Memorial University of Newfoundland
+Memorial University of Newfoundland and Labrador

scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git commit -m"Correcting university name"
[main a38ca54] Correcting university name
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Think of `git commit` as taking a snapshot of directory each time you run it, and `git add` is telling git what should be that snapshot. In this model, `git diff` tells us what we *could* include in the next snapshot, but haven't yet.

To see what is currently going to be in next snapshot, use `git diff --staged` (say these changes are already "staged" for commit).

As we make more commits, log gets longer. Can use `git log -N` to show last N commits, or `git log --oneline` to show just messages:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git log -2
commit a38ca548407d23258830dd2b61e4da12d3ae94a5 (HEAD -> main)
Author: Scott MacLachlan <smaclachlan@mun.ca>
Date:   Tue Sep 23 10:53:59 2025 -0230

    Correcting university name

commit 1413f064c3bfa01ead2afed259181dc47e36aa72
Author: Scott MacLachlan <smaclachlan@mun.ca>
Date:   Tue Sep 23 10:48:16 2025 -0230

    Adding title
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git log --oneline
a38ca54 (HEAD -> main) Correcting university name
1413f06 Adding title
48fdbd0 Adding university
69db617 Add myname.txt
```

These hexadecimal numbers are "git hashes" which we use to see changes between versions and other things

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$
diff --git a/myname.txt b/myname.txt
index 7c7d1b0..38bbd7f 100644
--- a/myname.txt
+++ b/myname.txt
@@ -1,3 +1,3 @@
-Scott MacLachlan
-Memorial University of Newfoundland
+Professor Scott MacLachlan
+Memorial University of Newfoundland and Labrador
```

(Note that order matters here, reversing commits gives inverted data. Generally want first hash to be the older data)

One thing to know is that git tracks files, but not directories:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ mkdir CMSC6950
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add CMSC6950
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
nothing to commit, working tree clean
```

Add a file in that directory:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ nano schedule.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ./

nothing added to commit but untracked files present (use "git add" to track)
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ git add schedule.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ git diff --staged
diff --git a/CMSC6950/schedule.txt b/CMSC6950/schedule.txt
new file mode 100644
index 0000000..61f1f80
--- /dev/null
+++ b/CMSC6950/schedule.txt
@@ -0,0 +1,2 @@
+HW 1 due Oct 2
+
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ git commit -m"Adding schedule"
[main 464bd40] Adding schedule
 1 file changed, 2 insertions(+)
 create mode 100644 CMSC6950/schedule.txt
```

Note: can run git commands in subdirectories of a repository and it identifies those with parent directory repository

As we collect more commits, helpful to have tools to look at the differences.

Call the most recent commit `HEAD` and can diff older commits relative to it

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ git diff HEAD~1 HEAD
diff --git a/CMSC6950/schedule.txt b/CMSC6950/schedule.txt
new file mode 100644
index 0000000..61f1f80
--- /dev/null
+++ b/CMSC6950/schedule.txt
@@ -0,0 +1,2 @@
+HW 1 due Oct 2
+
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ git diff HEAD~2 HEAD~1
diff --git a/myname.txt b/myname.txt
index 24415be..38bbd7f 100644
--- a/myname.txt
+++ b/myname.txt
@@ -1,3 +1,3 @@
 Professor Scott MacLachlan
-Memorial University of Newfoundland
+Memorial University of Newfoundland and Labrador

scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ git diff HEAD~2 HEAD
diff --git a/CMSC6950/schedule.txt b/CMSC6950/schedule.txt
new file mode 100644
index 0000000..61f1f80
--- /dev/null
+++ b/CMSC6950/schedule.txt
@@ -0,0 +1,2 @@
+HW 1 due Oct 2
+
diff --git a/myname.txt b/myname.txt
index 24415be..38bbd7f 100644
--- a/myname.txt
+++ b/myname.txt
@@ -1,3 +1,3 @@
 Professor Scott MacLachlan
-Memorial University of Newfoundland
+Memorial University of Newfoundland and Labrador

scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ git diff HEAD~2
diff --git a/CMSC6950/schedule.txt b/CMSC6950/schedule.txt
new file mode 100644
index 0000000..61f1f80
--- /dev/null
+++ b/CMSC6950/schedule.txt
@@ -0,0 +1,2 @@
+HW 1 due Oct 2
+
diff --git a/myname.txt b/myname.txt
index 24415be..38bbd7f 100644
--- a/myname.txt
+++ b/myname.txt
@@ -1,3 +1,3 @@
 Professor Scott MacLachlan
-Memorial University of Newfoundland
+Memorial University of Newfoundland and Labrador

scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$
```

Use `HEAD~N` for the Nth-to-last commit. If we're diffing against `HEAD`, don't need to specify it

Can also just diff one file:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ git diff HEAD~2 ../myname.txt
diff --git a/myname.txt b/myname.txt
index 24415be..38bbd7f 100644
--- a/myname.txt
+++ b/myname.txt
@@ -1,3 +1,3 @@
 Professor Scott MacLachlan
-Memorial University of Newfoundland
+Memorial University of Newfoundland and Labrador
```

To complement this, use `git show` to see changes that were made in a particular commit:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ git show HEAD~2
commit 1413f064c3bfa01ead2afed259181dc47e36aa72
Author: Scott MacLachlan <smaclachlan@mun.ca>
Date:   Tue Sep 23 10:48:16 2025 -0230

    Adding title

diff --git a/myname.txt b/myname.txt
index 7c7d1b0..24415be 100644
--- a/myname.txt
+++ b/myname.txt
@@ -1,3 +1,3 @@
-Scott MacLachlan
+Professor Scott MacLachlan
 Memorial University of Newfoundland
```

One advantage of storing all of this history is that we can recover old versions:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example/CMSC6950$ cd ..
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ cat myname.txt
Professor Scott MacLachlan
Memorial University of Newfoundland and Labrador

scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git checkout HEAD~2 myname.txt
Updated 1 path from fec97fb
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ cat myname.txt
Professor Scott MacLachlan
Memorial University of Newfoundland

scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   myname.txt
```

Note this shows changes to be committed!

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git diff --staged
diff --git a/myname.txt b/myname.txt
index 38bbd7f..24415be 100644
--- a/myname.txt
+++ b/myname.txt
@@ -1,3 +1,3 @@
 Professor Scott MacLachlan
-Memorial University of Newfoundland and Labrador
+Memorial University of Newfoundland
```

Discard the changes just by following directions:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git restore --staged myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   myname.txt

no changes added to commit (use "git add" and/or "git commit -a")
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git diff --staged
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git diff
diff --git a/myname.txt b/myname.txt
index 38bbd7f..24415be 100644
--- a/myname.txt
+++ b/myname.txt
@@ -1,3 +1,3 @@
 Professor Scott MacLachlan
-Memorial University of Newfoundland and Labrador
+Memorial University of Newfoundland

scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git restore myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ cat myname.txt
Professor Scott MacLachlan
Memorial University of Newfoundland and Labrador

scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$
```

2 steps here:

1. Unstage the changes. Go back to state with a modified file like before we ran `git add`
2. Restore the file from the current `HEAD` version of the repository

Suppose I make a bigger mistake and commit it.

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ nano myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add myname.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git commit -m"Add department"
[main 8ab3715] Add department
 1 file changed, 1 insertion(+)
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ nano CMSC6950/schedule.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add CMSC6950/schedule.txt
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git commit -m"Adding HW2 to schedule"
[main eeff0b8] Adding HW2 to schedule
 1 file changed, 1 insertion(+)
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git log --oneline
eeff0b8 (HEAD -> main) Adding HW2 to schedule
8ab3715 Add department
464bd40 Adding schedule
a38ca54 Correcting university name
1413f06 Adding title
48fdbd0 Adding university
69db617 Add myname.txt
```

Could either recover `myname.txt` from before `HEAD~2` or *revert* that commit

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git revert 8ab3715
[main 61ec659] Revert "Add department"
 1 file changed, 1 deletion(-)
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
nothing to commit, working tree clean
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ cat myname.txt
Professor Scott MacLachlan
Memorial University of Newfoundland and Labrador

scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git log --oneline
61ec659 (HEAD -> main) Revert "Add department"
eeff0b8 Adding HW2 to schedule
8ab3715 Add department
464bd40 Adding schedule
a38ca54 Correcting university name
1413f06 Adding title
48fdbd0 Adding university
69db617 Add myname.txt
```

A common issue is when we start to have files in our directory that we don't want to track. For example, if we're writing code that we compile, don't want to store output from the compilation process (since that's specific to processors and compilers)

Example: suppose you compile your code and this produces a log file:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ nano compile.log
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git statu
git: 'statu' is not a git command. See 'git --help'.

The most similar commands are
        status
        stage
        stash
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        compile.log

nothing added to commit but untracked files present (use "git add" to track)
```

git "sees" the file and commands like `git commit -a -m"..."` will add the file to the repo. This makes it hard to work with the repo, because every time somebody compiles the code, it shows changes in the repo, even if the code itself doesn't change.

Solution is to add a `.gitignore` file to the directory:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ nano .gitignore
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git statu
git: 'statu' is not a git command. See 'git --help'.

The most similar commands are
        status
        stage
        stash
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)
```

Note: as soon as we edit `.gitignore`, it impacts what git "sees" as files that should be tracked.

We store this file in the repo:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add .gitignore
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git commit -m"Adding gitignore"
[main 9c39f5d] Adding gitignore
 1 file changed, 4 insertions(+)
 create mode 100644 .gitignore
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
nothing to commit, working tree clean
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ ls
CMSC6950  compile.log  myname.txt
```

Try to add `compile.log` anyways:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add compile.log
The following paths are ignored by one of your .gitignore files:
compile.log
hint: Use -f if you really want to add them.
hint: Turn this message off by running
hint: "git config advice.addIgnoredFile false"
```

Can still add these files, but have to use `-f` to force it.

Can see what's being ignored by:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status --ignored
On branch main
Ignored files:
  (use "git add -f <file>..." to include in what will be committed)
        compile.log

nothing to commit, working tree clean
```

Can add exceptions to `.gitignore` with `!file`:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ nano data.log
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ nano .gitignore
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        data.log

no changes added to commit (use "git add" and/or "git commit -a")
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add .gitignore
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git commit -m"Update gitignore"
[main 9024c40] Update gitignore
 1 file changed, 1 insertion(+)
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git add data.log
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git commit -m"Adding data"
[main 48ff762] Adding data
 1 file changed, 5 insertions(+)
 create mode 100644 data.log
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git status
On branch main
nothing to commit, working tree clean
```

So far, we've worked with git locally, keeping the repository in the same place as our working copy. The real power of version control comes up when we store the repository remotely, and can access from many places.

Usual practice is to have a single cetnral repository (e.g., on github) and link this to a local repository that you work on.

- Each contributor has their own local copy of the repository where they will make changes that they "push" to the central repository.

To do this, we need to link our local repo to the central one, which we need to create.

Step 1: Create a remote repository.

Go to github, click "new"

- several options here that we'll mostly ignore for now, name repository to match local directory
- see options for "quick setuP", "new repo at command line", "push an existing repo"

Back to my command line to push existing repo:

```
scottmac@kaczmarz:~/CMSC6950/CMSC6950_Fall2025_example$ git remote add origin git@github.com:MacLachlanTeaching/CMSC6950_Fall2025_example.git
git branch -M main
git push -u origin main
Enumerating objects: 34, done.
Counting objects: 100% (34/34), done.
Delta compression using up to 16 threads
Compressing objects: 100% (23/23), done.
Writing objects: 100% (34/34), 3.14 KiB | 229.00 KiB/s, done.
Total 34 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), done.
To github.com:MacLachlanTeaching/CMSC6950_Fall2025_example.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Back to github, click "code" to see that we've synced.

IMPORTANT: Connection to github relies on "ssh keys". These are authentication credentials that take the place of passwords.

ssh keys come in pairs, a "private key" and a "public key"

You keep the private key on your system

- new key pair for each system that you use (i.e., from which you want to access github or other repositories or systems)
- give public key(s) to the sites that you want to access

Two steps:

1. Generate a key pair
2. Upload your public key into your github profile.

Do this. On HW2